AIMERZ.ai
Aim.Act.Achieve

NOTES

Encapsulation and Abstraction

Encapsulation and Abstraction

Encapsulation

What is Encapsulation?

Encapsulation refers to the bundling of data (attributes) and methods (functions) that operate on that data within a single unit or class. This principle restricts direct access to certain attributes and methods of an object, thereby protecting the object's internal state and preventing unintended modifications.

Why Encapsulation?

Data Protection: By using private members, encapsulation prevents data from being accessed or modified accidentally or maliciously. Sensitive information can be protected inside the class.
Example: In a BankAccount class, you would not want external code to modify the account balance directly without using a valid method to ensure proper transactions (like withdrawals or deposits).

Controlled Access: Encapsulation allows controlling how data is accessed or modified. You can define methods that validate or process the data before changing the internal state.
Example: In the deposit() method of the BankAccount class, you can validate that the deposit amount is positive before updating the balance.

Improved Code Maintenance: Encapsulation helps in changing and refactoring the internal implementation of a class without affecting

external code. Public methods provide a consistent interface, while the internal workings of the class can be changed as needed.

Example: You might later decide to store the balance in a different format or use encryption, but the public get_balance() method can still provide access to the balance without changing the external interface.

How Encapsulation Works?

Encapsulation in Python is primarily achieved using access modifiers, which control the visibility of class members (attributes and methods).

Access Modifiers in Python

Access modifiers in Object-Oriented Programming (OOP) control the visibility of class members (attributes and methods). They determine whether a member can be accessed from outside the class or if it's restricted to within the class itself. Python has three levels of access control: **Public**, **Protected**, and **Private**.

Public Access Modifier

- **Definition**: A public member is accessible from anywhere — both within and outside the class. By default, all class members in Python are public unless explicitly stated otherwise.
- **Syntax**: No special syntax is required for public members.
- **Use Case**: Public members provide an interface to the class, allowing other code to interact with the objects of that class.

Example:

```python
class Car:
    def __init__(self, brand, model):
        self.brand = brand  # Public attribute
        self.model = model  # Public attribute

    def display_info(self):  # Public method
        print(f"Brand: {self.brand}, Model: {self.model}")

# Creating an object of Car
car1 = Car("Tesla", "Model S")

# Accessing public attributes and method
print(car1.brand)  # Output: Tesla
print(car1.model)  # Output: Model S
car1.display_info()  # Output: Brand: Tesla, Model: Model S
```

Explanation of the Example

1. **Class Declaration:** The Car class is declared without any access modifier, making it public by default in Python (Python does not enforce access modifiers strictly like some other languages).
2. **Public Attributes:** The attributes brand and model are public. They can be accessed directly from outside the class using the object car1.
3. **Public Method:** The method display_info() is also public, allowing it to be called from outside the class to display information about the car.
4. **Object Creation:** An object car1 is created from the Car class with specific values for its attributes. The attributes and methods are accessed directly using this object.

Importance of Public Access Modifier

- **Flexibility:** Public members provide flexibility for other classes to interact with and utilise functionalities without needing to know the internal workings of those classes.
- **Ease of Use:** By exposing necessary methods and attributes as public, developers can create intuitive interfaces for their modules, making them easier to use.
- **Encapsulation Balance:** While providing access through public members, it's essential to balance encapsulation principles by keeping sensitive data private or protected when necessary.

Protected Access Modifier

- **Definition**: A protected member can be accessed within the class and by its subclasses, but it's conventionally **hidden** from external access. In Python, protected members are prefixed with a single underscore (_). While this doesn't technically enforce protection (since it's only a convention), it signals that these members are meant for internal use.
- **Syntax**: Prefix the attribute or method with a single underscore (_).
- **Use Case**: Protected members are often used when designing classes that will be extended via inheritance, where child classes need access to certain members but external code shouldn't interact with them directly.

Example:

```python
class Employee:
    def __init__(self, name, salary):
        self._name = name       # Protected attribute
        self._salary = salary   # Protected attribute

    def display_info(self):  # Public method
        print(f"Name: {self._name}, Salary: {self._salary}")

# Subclass extending Employee
class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department  # Public attribute

    def display_manager_info(self):
        print(f"Manager: {self._name}, Department:
{self.department}")

# Creating an object of Manager
mgr = Manager("Alice", 90000, "HR")

# Accessing protected and public attributes
mgr.display_info()  # Output: Name: Alice, Salary: 90000
mgr.display_manager_info()  # Output: Manager: Alice, Department: HR

# Accessing protected attribute directly (not recommended)
print(mgr._name)  # Output: Alice
```

Explanation of the Example

1. **Class Declaration:** The Employee class is defined with two protected attributes: _name and _salary. The single underscore prefix indicates that these attributes should be treated as protected.
2. **Constructor Method (__init__):** The constructor initialises the _name and _salary attributes when a new instance of Employee is created.

3. **Public Method:** The display_info() method is a public method that prints the employee's name and salary. It accesses the protected attributes directly, demonstrating that they can be used within the class.

Subclass Declaration: Manager

4. **Subclassing:**
   - The Manager class extends the Employee class, inheriting its properties and methods.
   - In its constructor, it calls super().__init__(name, salary) to initialise the inherited attributes.
5. **Accessing Protected Attributes in Subclass:** The display_manager_info() method accesses _name, which is a protected attribute inherited from Employee. This demonstrates how subclasses can access protected members of their parent classes.
6. **Creating an Instance:** An instance of Manager named mgr is created with specific values for name, salary, and department.
7. **Calling Methods:**
   - The public method display_info() is called on mgr, which outputs Alice's name and salary.
   - The method display_manager_info() is called to show additional information about Alice as a manager.

Direct Access to Protected Attributes

8. **Accessing Protected Attributes Directly:** Although it's possible to access _name directly from outside the class (as shown), this practice is generally discouraged. The underscore prefix serves

as a convention indicating that this attribute is intended for internal use within the class hierarchy.

Benefits of Using Protected Access Modifier

- **Controlled Visibility:** By using protected attributes, you allow subclasses to access necessary data while preventing external code from modifying it directly.
- **Encapsulation with Flexibility:** Protected members provide encapsulation benefits while still allowing derived classes to extend or modify behaviour based on shared state.
- **Clearer Intentions:** Using a single underscore signals to developers that certain members are not part of the public API and should be used cautiously.

Private Access Modifier

- **Definition**: A private member is only accessible within the class in which it is defined. It cannot be accessed or modified from outside the class or by subclasses. In Python, private members are prefixed with double underscores (__).
- **Syntax**: Prefix the attribute or method with double underscores (__).
- **Use Case**: Private members are used to encapsulate the internal state of a class, ensuring that certain attributes or methods can't be altered or accessed from outside the class, providing data security and integrity.

Example:

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner            # Public attribute
        self.__balance = balance      # Private attribute

    def get_balance(self):  # Public method to access private attribute
        return self.__balance

    def deposit(self, amount):  # Public method to modify private attribute
        if amount > 0:
            self.__balance += amount
        else:
            print("Deposit amount must be positive.")

# Creating an object of BankAccount
account = BankAccount("John", 1000)

# Accessing public attribute
print(account.owner)  # Output: John

# Accessing private attribute directly (will raise an error)
# print(account.__balance)  # AttributeError: 'BankAccount' object has no attribute '__balance'

# Accessing private attribute using a public method
print(account.get_balance())  # Output: 1000

# Modifying private attribute using a public method
account.deposit(500)
print(account.get_balance())  # Output: 1500
```

Explanation of the Example

1. **Class Declaration:** BankAccount class is defined with one public attribute (owner) and one private attribute (__balance). The double underscore prefix (__) indicates that __balance is intended to be private.

2. **Constructor Method (__init__):** The constructor initialises the owner and __balance attributes when a new instance of BankAccount is created.
3. **Public Methods:**
    - The get_balance() method is a public method that allows external code to access the private __balance attribute safely.
    - The deposit(amount) method allows external code to modify the private __balance attribute while enforcing rules (e.g., only allowing positive amounts).
4. **Creating an Instance:** An instance of BankAccount named account is created with specific values for owner and initial balance.
5. **Accessing Public Attributes:** The public attribute owner can be accessed directly.
6. **Attempting Direct Access to Private Attributes:** If you attempt to access __balance directly (commented out in the example), it will raise an AttributeError. This demonstrates that private attributes cannot be accessed from outside the class.
7. **Accessing Private Attributes via Public Methods:** The private attribute can be accessed using the public method.
8. **Modifying Private Attributes via Public Methods:** The deposit() method allows for controlled modification of the private balance.

Benefits of Using Private Access Modifier

- **Data Protection:** By keeping sensitive data like account balances private, developers can prevent unauthorised access and modifications, enhancing security.
- **Control Over Data Manipulation:** Public methods can enforce rules and validations when accessing or modifying private attributes, ensuring that only valid operations are performed.
- **Improved Maintainability:** Changes to private members can be made without affecting external code, as long as public interfaces remain consistent.

Why Use Access Modifiers?

1. **Data Protection**: By making certain attributes and methods private, you ensure that critical parts of the class are not accidentally modified from outside the class, protecting the internal state.
2. **Controlled Access**: Public and protected members allow controlled interaction with class attributes, offering methods (like getters and setters) to validate inputs or perform necessary operations before changing values.
3. **Maintainability**: Using access modifiers makes the code easier to maintain. Developers working on a class can ensure that only necessary members are exposed, preventing unwanted interference from other parts of the program.

Getters and Setters

What:

Getters and setters are special methods used to access (get) or modify (set) private attributes of a class. Private attributes are variables that are not meant to be accessed directly from outside the class. Getters retrieve the value of these attributes, while setters allow you to modify them, often with added validation.

Why:

Getters and setters provide a controlled way to access and update private attributes, ensuring data encapsulation. By using them, you can enforce rules, such as preventing negative values or invalid data, before modifying the attributes. This helps protect the internal state of the object from unauthorised or inappropriate changes.

- **Getter:** A method that returns the value of a private attribute.
- **Setter:** A method that updates the value of a private attribute, typically with some validation logic.

In Python, private attributes are usually denoted by a double underscore (__) prefix.

Example 1:

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def get_balance(self):  # Getter
        return self.__balance

    def set_balance(self, amount):  # Setter
```

```
        if amount >= 0:
            self.__balance = amount
        else:
            print("Invalid amount")

account = BankAccount(1000)
print(account.get_balance())  # Output: 1000

account.set_balance(500)
print(account.get_balance())  # Output: 500

account.set_balance(-200)  # Output: Invalid amount
```

The BankAccount class has a private attribute __balance that cannot be accessed directly from outside the class. Instead, we use the get_balance() method to retrieve the balance, and the set_balance() method to update it. The setter includes a validation step, ensuring that the balance cannot be set to a negative value. This enforces proper data management and prevents invalid states.

Example 2:

Python

```
class Temperature:
    def __init__(self, celsius):
        self.__celsius = celsius  # Private attribute

    def get_celsius(self):  # Getter
        return self.__celsius

    def set_celsius(self, value):  # Setter
        if value >= -273.15:  # Absolute zero check
            self.__celsius = value
        else:
            print("Temperature cannot be below absolute zero")

temp = Temperature(25)
print(temp.get_celsius())  # Output: 25
```

```
temp.set_celsius(100)
print(temp.get_celsius())  # Output: 100

temp.set_celsius(-300)  # Output: Temperature cannot be below
absolute zero
```

In this `Temperature` class, the `__celsius` attribute represents temperature in Celsius. The `get_celsius()` method retrieves the value of the temperature, while the `set_celsius()` method allows the value to be updated but ensures it cannot go below absolute zero (-273.15°C). This validation ensures that the object's state remains logical and valid.

Benefits of Using Methods, Getters, and Setters

- **Encapsulation:** Methods, getters, and setters are key to the principle of **encapsulation** in object-oriented programming. They allow you to hide the internal implementation details of the object and expose only necessary parts through well-defined interfaces.
- **Controlled Access:** By using getters and setters, you provide a controlled way for interacting with the internal state of an object. This is especially important when working with complex systems where certain rules must be followed for maintaining consistency and correctness.
- **Reusability:** Once defined, methods can be reused across different parts of the program. This makes the code cleaner and more organised, avoiding redundancy and enabling better code maintenance.

Abstraction

Abstraction is one of the key principles of Object-Oriented Programming (OOP) and focuses on hiding the complexity of the system while exposing only the essential features or functions. The main goal of abstraction is to allow users to interact with an object at a higher level, without needing to know the intricate details of how things work internally.

For instance, think of a car's "accelerate" feature. When you press the accelerator pedal, the car increases its speed. You don't need to know how the engine injects fuel, adjusts air intake, or alters the combustion process. You simply use the "accelerate" function. Similarly, in Python, abstraction allows developers to create complex operations that users can easily interact with, without understanding the internal mechanics.

In Python, abstraction can be achieved through **abstract classes** and **interfaces** (although Python doesn't have true interfaces like some other languages, it uses abstract classes to achieve similar functionality).

How Abstraction Works?

1. **Abstract Classes**:
    - An **abstract class** cannot be instantiated directly. It is designed to be a base class that defines methods but leaves the implementation of those methods to derived classes (subclasses).
    - Abstract classes use methods that are declared but have no concrete implementation. Subclasses must override

and provide specific implementations for these abstract methods.

- Python provides the `ABC` (Abstract Base Class) module from the `abc` library to create abstract classes and methods.

2. **Abstract Methods**:
   - Abstract methods are declared in the abstract class and must be implemented by any subclass that inherits from the abstract class.
   - If a subclass does not implement all the abstract methods, it will also become an abstract class.

Why Use Abstraction?

- **Simplification**: Abstraction simplifies the complex system by breaking it down into smaller, more manageable parts.
- **Standardisation**: Abstract classes provide a blueprint that ensures subclasses implement specific methods, maintaining consistency across different implementations.
- **Loose Coupling**: By interacting with objects at a higher level of abstraction, you reduce dependency on the specific details of an implementation, which can change without affecting the code using it.

Abstract Base Classes (ABC)

What is an ABC?

Abstract Base Classes (ABCs) provide a blueprint for other classes, ensuring consistency across different implementations. An ABC is a class that cannot be instantiated on its own and is meant to be inherited by other classes. It often defines methods (using the @abstractmethod decorator) that subclasses must implement, setting a clear expectation for subclasses without dictating their specific functionality. In Python, ABCs are created using the abc module, which allows us to enforce a structured design within our program.

Why Use ABCs?

ABCs ensure that subclasses share a common structure, making it easier to work with various classes interchangeably if they share a set of required methods. By using ABCs, developers can enforce that all subclasses implement specific methods, which is particularly helpful when designing complex applications where certain functionalities must exist in every subclass.

Example:

```python
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def sound(self):
```

```
        pass
# Any subclass of Animal must implement the 'sound' method
class Dog(Animal):
    def sound(self):
        return "Woof!"

class Cat(Animal):
    def sound(self):
        return "Meow!"

# Using the subclasses
dog = Dog()
cat = Cat()
print(dog.sound())  # Outputs: Woof!
print(cat.sound())  # Outputs: Meow!
```

In this example, the Animal class is an ABC with an abstract method sound. Any class that inherits from Animal must implement the sound method, ensuring that all animal subclasses have a way to make a sound.

@abstractmethod Decorator

Purpose of @abstractmethod:

The @abstractmethod decorator in Python is used to define abstract methods within an ABC. An abstract method is a method that has a declared name and expected behaviour but no actual implementation. This means that any subclass inheriting from an ABC must provide its own implementation of the abstract method, ensuring consistent behaviour across different subclasses.

How to Use?

To declare a method as abstract, simply place the @abstractmethod decorator above the method definition in the base class. Subclasses are then obligated to implement these abstract methods; otherwise, Python will raise an error when you try to instantiate them.

Example:

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

circle = Circle(5)
print(circle.area())  # Outputs the area of the circle.
```

Here, the Shape class defines an abstract area method. The Circle class then implements this method, providing a specific formula to calculate the area.

Example of Abstraction Using Abstract Classes

We use an abstract class to model different types of vehicles.

Python

```python
from abc import ABC, abstractmethod

# Abstract Class
class Vehicle(ABC):

    @abstractmethod
    def start_engine(self):
        pass

    @abstractmethod
    def stop_engine(self):
        pass

    def fuel_type(self):
        print("Fuel type depends on the vehicle.")

# Concrete Class (Subclasses that implement the abstract methods)
class Car(Vehicle):

    def start_engine(self):
        print("Car engine started.")

    def stop_engine(self):
        print("Car engine stopped.")

    def fuel_type(self):
        print("Car uses gasoline.")

class ElectricCar(Vehicle):

    def start_engine(self):
        print("Electric car powered on.")

    def stop_engine(self):
        print("Electric car powered off.")

    def fuel_type(self):
        print("Electric car uses electricity.")

# Trying to instantiate the abstract class will raise an error
```

```
# vehicle = Vehicle()  # TypeError: Can't instantiate abstract class

# Instantiate concrete subclasses
car = Car()
car.start_engine()  # Output: Car engine started.
car.fuel_type()     # Output: Car uses gasoline.
car.stop_engine()   # Output: Car engine stopped.

# Instantiate electric car
electric_car = ElectricCar()
electric_car.start_engine()  # Output: Electric car powered on.
electric_car.fuel_type()      # Output: Electric car uses
electricity.
electric_car.stop_engine()    # Output: Electric car powered off.
```

Abstract Class: Vehicle

- The Vehicle class is defined as an abstract class by inheriting from ABC (Abstract Base Class).
- It contains two abstract methods: start_engine() and stop_engine(), which must be implemented by any subclass.
- The method fuel_type() is a concrete method that provides a default implementation, indicating that the fuel type depends on the vehicle.

Concrete Classes: Car and ElectricCar

- **Car:** This class inherits from Vehicle and implements the abstract methods:
    - start_engine(): Prints a message indicating that the car's engine has started.

- stop_engine(): Prints a message indicating that the car's engine has stopped.
- fuel_type(): Overrides the default implementation to specify that the car uses gasoline.

- **ElectricCar:** This class also inherits from Vehicle and implements the abstract methods:
  - start_engine(): Prints a message indicating that the electric car is powered on.
  - stop_engine(): Prints a message indicating that the electric car is powered off.
  - fuel_type(): Overrides the default implementation to specify that the electric car uses electricity.

Instantiation and Method Calls

- The line vehicle = Vehicle() is commented out because attempting to instantiate an abstract class will raise a TypeError.
- Instances of the concrete classes are created:
  - An instance of Car is created, and its methods are called, demonstrating how it starts, stops, and identifies its fuel type.
  - An instance of ElectricCar is created, with similar method calls to show its functionality.

Output:

```
Python
Car engine started.
Car uses gasoline.
Car engine stopped.
Electric car powered on.
Electric car uses electricity.
Electric car powered off.
```

This output confirms that:

- The methods for both concrete classes are functioning as intended.
- Each subclass provides its specific implementation for starting and stopping the engine, as well as specifying the fuel type.

Key Benefits of Abstraction

1. **Focus on Essentials**: With abstraction, users can interact with the important functionalities of an object, without needing to know the inner workings.
2. **Modularity**: Abstraction helps in building a system that is modular, where different parts of the system can be worked on independently.
3. **Code Reusability**: Abstract classes provide a template that can be reused across various subclasses, ensuring consistency and reducing redundancy in code.
4. **Implementation Flexibility**: While the abstract class defines the structure, different subclasses can implement the same methods in different ways based on their specific requirements.
5. **Separation of Concerns**: Abstraction allows developers to separate the concerns between what a class does (as defined

by abstract methods) and how it does it (as defined by the subclasses).

Practical Uses of Abstraction

Abstraction is widely used in data science, software development, and systems architecture to simplify complex processes. For example, in a data science pipeline, users often interact only with high-level methods such as load_data or train_model without needing to see the code behind them. This approach allows data scientists to focus on analysis and model tuning rather than preprocessing, feature engineering, or algorithm optimization, which are abstracted away into reusable functions or classes.

By hiding complexity, abstraction improves readability, reduces cognitive load, and ensures that users only work with what they need. This separation also makes code more modular, allowing components to be updated independently of one another. For instance, a Car class could have various functionalities like start, stop, and accelerate, but changes to any of these implementations would not affect how the user interacts with them.

Abstract Classes and Interfaces in Python

Difference Between Abstract Class and Interface:

In many programming languages, interfaces are specific constructs used to define methods that must be implemented in any implementing class. In Python, interfaces aren't a standalone construct, but we use ABCs to achieve similar functionality. Abstract

classes in Python can contain both concrete methods (methods with an implementation) and abstract methods, whereas an interface typically contains only method signatures (in languages with a dedicated interface construct).

An abstract class allows for both defining required methods and providing shared functionality, while an interface strictly defines a contract. Python's flexibility means ABCs are used to mimic interface behaviour, combining both structural enforcement and optional shared methods.

How Interfaces Are Used in Python?

In Python, using an ABC with @abstractmethod functions as an interface. This enforces that any subclass implementing this ABC must provide its version of these methods, ensuring a consistent structure and behaviour across subclasses.

Example:

```python
from abc import ABC, abstractmethod

class Dataset(ABC):
    @abstractmethod
    def load_data(self):
        pass

class CSVData(Dataset):
    def load_data(self):
        print("Loading data from a CSV file")

class APIData(Dataset):
```

```
    def load_data(self):
        print("Loading data from an API")

# Both subclasses implement the load_data method as required by the
Dataset interface.
csv_data = CSVData()
csv_data.load_data()  # Outputs: Loading data from a CSV file
```

Here, Dataset functions as an interface by defining an abstract load_data method. Subclasses like CSVData and APIData must implement this method, creating consistency for any data-loading classes derived from Dataset.

Using ABCs and abstraction principles, Python developers can design more maintainable, readable, and extensible code. The combination of ABCs and @abstractmethod provides a structured approach, encouraging consistency and modularity across complex applications.

THANK YOU

AIMERZ.ai
Aim.Act.Achieve